

**INTERNATIONAL CYBERSECURITY
AND DIGITAL FORENSICS ACADEMY**

**INTRO TO CRITICAL INFRASTRUCTURE
FOR IT LEARNERS**

LAB 06 — INTRODUCTION TO MODBUS

Student Edition

Authorized ICDFA training environments only
icdfa.edu.ng

Lab 06: Introduction to Modbus

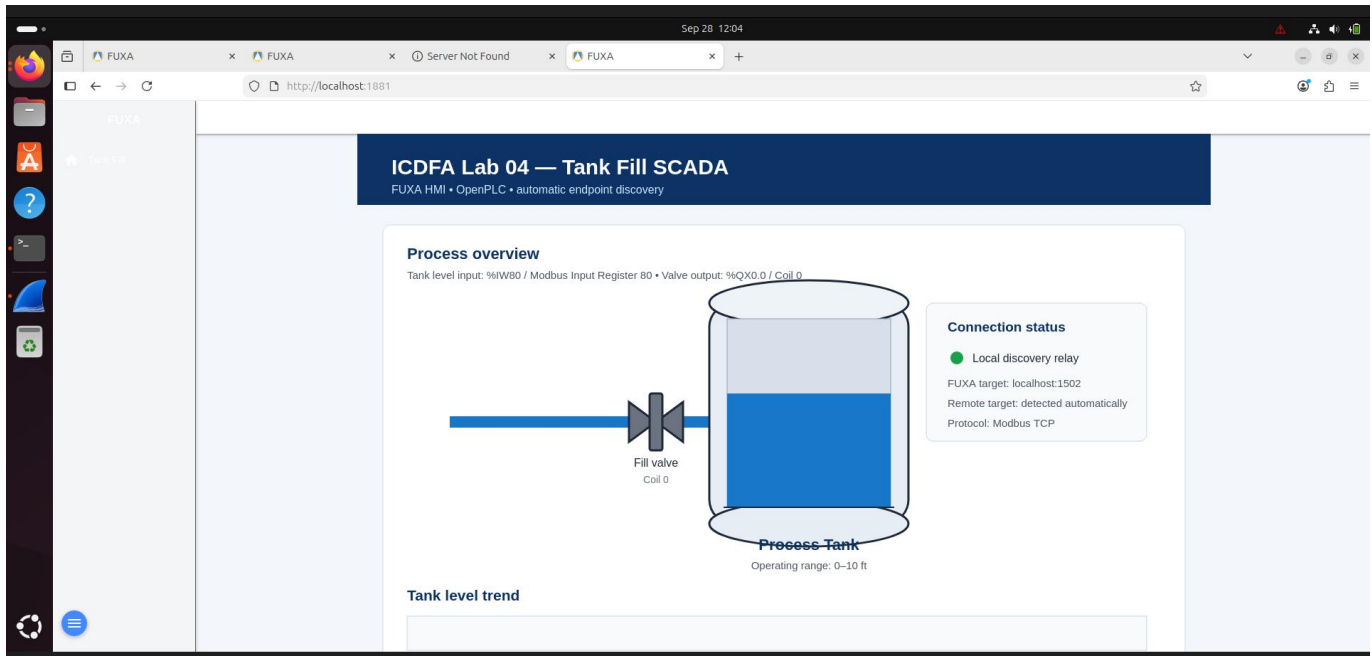
Course: Intro to Critical Infrastructure for IT Learners | **Estimated Duration:** 3–4 hours

Procedure

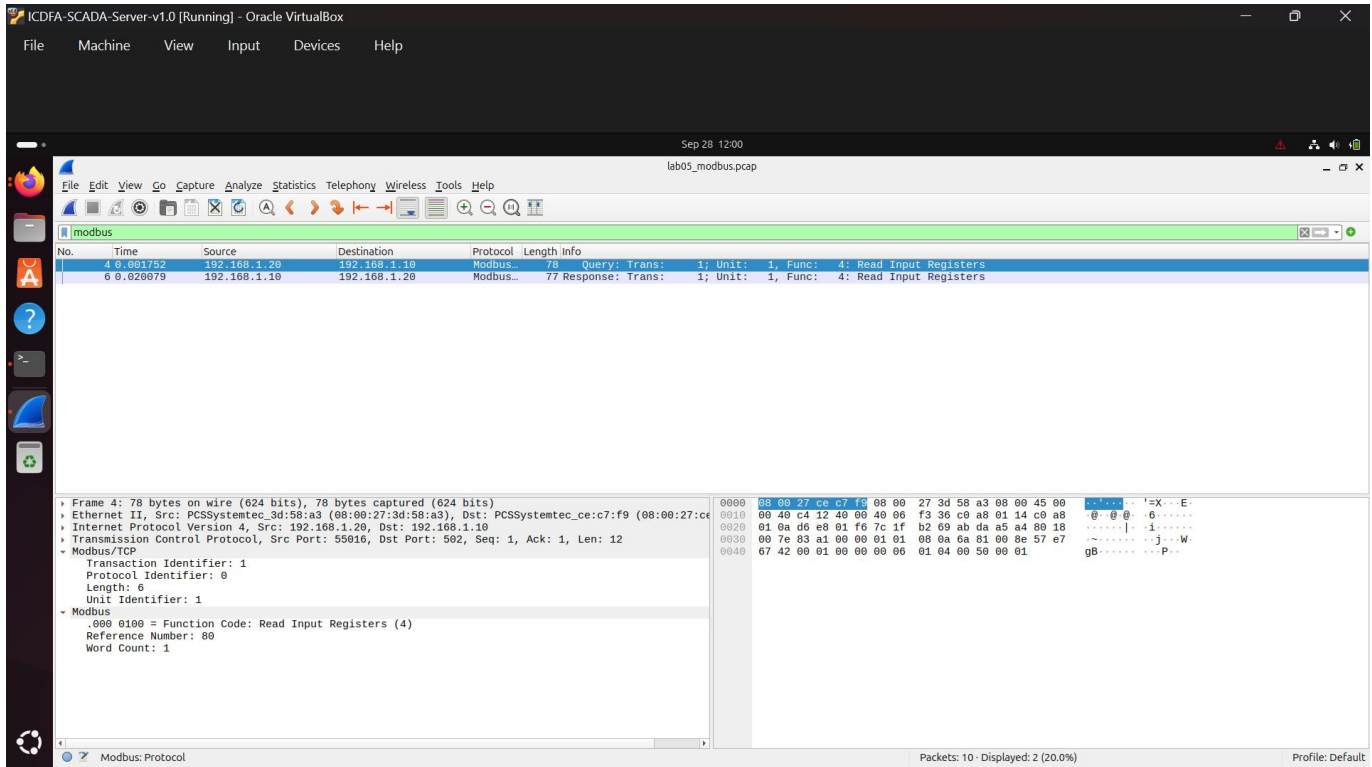
- Step 1.** Start the PLC, SCADA and analyst VMs on the instructor-provided isolated laboratory topology. Do not connect this lab to production or bridged networks.
- Step 2.** On the PLC VM, start the TankFillSim program in OpenPLC Runtime and confirm Running status.
- Step 3.** On the SCADA VM, start FUXA, load the tank-fill HMI and confirm the openplc connection targets 192.168.1.10:502.
- Step 4.** Run the HMI and verify that the tank level changes between its operating limits.
- Step 5.** On the analyst VM, open Wireshark. Select only the interface connected to the ICDFA lab segment carrying PLC/SCADA traffic.
- Step 6.** Apply the display filter modbus. If needed, refine it with `tcp.port == 502`.
- Step 7.** Start a short packet capture while the HMI is running, allow several process updates to occur, then stop the capture.
- Step 8.** Select a Modbus request/response pair and expand the Modbus/TCP and Modbus protocol fields.
- Step 9.** Identify the transaction/request, source and destination IP addresses, function code and referenced register/data value.
- Step 10.** Locate a Function Code 4 (Read Input Registers) response associated with the tank-level value. In the teaching image, Input Register 80 corresponds to the Tank Level indication.
- Step 11.** Complete the Modbus function-code table by explaining at least codes 01, 02, 03, 04, 05, 06, 15 and 16.
- Step 12.** Save the packet capture with a filename containing your registration number and LAB06. Do not capture unrelated traffic.

Required Evidence & Submission

- Screenshot of the FUXA/OpenPLC process running.



- **Screenshot of Wireshark filtered to Modbus traffic.**



- **One annotated request/response analysis identifying function code and register.**

Request Frame Analysis (Frame #4)

- Source IP / Destination IP: 192.168.1.20 -> 192.168.1.10
- Transport: TCP Port 55016 -> Port 502 (mbap)
- MBAP Header:
 - Transaction Identifier: 1 (0x0001)
 - Protocol Identifier: 0 (0x0000, Modbus TCP)
 - Length: 6 bytes
 - Unit Identifier: 1 (0x01)
- Modbus PDU (Protocol Data Unit):
 - Function Code: Read Input Registers (4)
 - Reference Number (Register Address): 80 (0x0050, mapped to PLC input register %IW80)
 - Word Count (Quantity): 1 (requests one 16-bit input register)

Response Frame Analysis (Frame #6)

- Source IP / Destination IP: 192.168.1.10 -> 192.168.1.20
- MBAP Header: Matches Transaction ID 1 and Unit ID 1 to pair with Query #4.

- Modbus PDU:
 - Function Code: Read Input Registers (4)
 - Byte Count: 2 bytes (1 word = 16 bits = 2 bytes)
 - Register Data: Returns the current 16-bit integer value held in %IW80.
- **A completed function-code table and a short comparison of Modbus TCP versus Modbus RTU.**

Modbus TCP vs. Modbus RTU

Characteristic	Modbus TCP	Modbus RTU
Physical Layer	Ethernet (IEEE 802.3, RJ-45)	Serial (RS-485, RS-232, or RS-422)
Header / Framing	7-byte MBAP Header appended to PDU	1-byte Slave Address + PDU + 2-byte CRC
Error Checking	Handled at Ethernet & TCP transport layers	Explicit 16-bit CRC appended to every serial frame
Addressing	IP Address + 1-byte Unit ID	1-byte Slave ID (1–247)
Network Topology	Star / Switch-based Ethernet topology	Daisy-chained serial bus topology
Standard Port	TCP Port 502	N/A (Baud rate / Parity bit settings)

Knowledge Check

1. What transport/networking difference separates Modbus TCP from Modbus RTU?

Response: Modbus TCP operates over Ethernet networks using TCP/IP (typically on Port 502) with a 7-byte MBAP header, whereas Modbus RTU runs over serial communications (RS-485, RS-232, or RS-422) using a raw binary frame protected by a 16-bit CRC error check.

2. Why does Modbus use function codes?

Response: Modbus uses function codes within the Protocol Data Unit (PDU) to instruct the slave/server device on what specific action to perform (e.g., reading or writing) and which memory data type to target (discrete inputs, coils, input registers, or holding registers).

3. What does Function Code 4 request/read?

Response: Function Code 4 requests to read 16-bit read-only Input Registers from the PLC (such as analog sensor inputs like process tank levels or temperature readings, mapped to %IW registers).

4. Why is visibility of unprotected industrial protocol traffic a cybersecurity concern?

Response: Because Modbus TCP lacks native encryption and authentication, unencrypted traffic allows attackers to sniff operational parameters, map the control system architecture, and perform unauthorized command injection or man-in-the-middle attacks to tamper with industrial processes.